

Interacting with Student Information on Google Sheets Using the RAG Technique

Chacharin Lertyosbordin
King Mongkut's University
of Technology Thonburi
Bangkok, Thailand
lertyosbordin@outlook.com

Nattawat Panyapakdeewong
Bangkok Christian College
Bangkok, Thailand
nattawatfrank2008@gmail.com

Panyakorn Singhadoung
The Demonstration School of
Bansomdejchaopraya Rajabhat University
Bangkok, Thailand
hspg.pan@gmail.com

Abstract—This study presents an AI-powered chatbot that enables natural language access to student records stored in Google Sheets. The system integrates Retrieval-Augmented Generation (RAG) using Gemini 1.5 Flash and ChromaDB for semantic embedding, vector-based retrieval, and contextual response generation. A RESTful API, developed using Python Flask, supports integration with external platforms, allowing users to query academic, health, contact, and demographic data. Developed under the Design and Development Research (DDR), Type I model, the prototype was tested with 15 representative queries against a mock dataset of 35 students. Results showed average token usage of 316.5 for prompts and 37.7 for responses, with 14 of 15 answers receiving the highest relevance score. The findings demonstrate that a lightweight, RAG-based chatbot can offer efficient and accurate educational data access, supporting non-technical users in retrieving information without direct spreadsheet interaction.

Keywords — Chatbot, Inventory, RAG

I. INTRODUCTION

A. Background

Managing student information, including grades, attendance, health records, and contact details, is essential but often labor-intensive in school settings. Traditionally, this data is stored in Student Information Systems or even simple spreadsheets (e.g. Google Sheets), requiring teachers or staff to locate records by browsing spreadsheets or applying complex queries to find specific details. This creates a barrier for educators and administrators who may not be proficient with database queries or spreadsheet formulas. The longstanding vision of natural language interfaces to data is to eliminate this barrier, enabling users to retrieve and interact with data using natural language [1].

In an educational context, a natural language access to student records could empower teachers, school staff, and even students to pose queries such as ‘Which student has the highest GPA in mathematics this term?’ or ‘What is the contact number of Nattawat’s parent?’ and receive prompt responses without requiring technical expertise. Recent advances in Large Language Models (LLMs) have made such conversational data access feasible at scale [2]. LLMs can interpret nuanced natural language questions; however, a standalone LLM has no direct knowledge of a school’s private Google Sheet and may generate

inaccurate or fabricated responses when asked about specific student data.

To reliably ground the model in the actual records, our system employs Retrieval-Augmented Generation (RAG). RAG is an AI framework that combines traditional information retrieval with generative AI [3]. Instead of relying solely on the LLM’s pre-trained knowledge, the model first retrieves relevant data from an external source (in this case, the Google Sheets student dataset) and then augments the model’s input with this data before generating a response.

Our system integrates the RAG technique through a combination of specialized tools. On the retrieval side, we use ChromaDB as a vector database to index and search the student information semantically. Each student record (or semantically relevant segment of data from the Google Sheet) is converted into an embedding – a numeric representation capturing its meaning. Vector-based retrieval allows the system to find entries that are conceptually relevant to a user’s question, even if the wording doesn’t exactly match. This is in line with modern RAG pipelines where documents are stored as high-dimensional embeddings to support efficient semantic retrieval [4].

For the generative component, we use Google’s Gemini 1.5 Flash model as the LLM. Gemini 1.5 Flash is a state-of-the-art multimodal LLM known for its speed and efficiency, offering a long context window and strong reasoning abilities despite being a lightweight model [5]. These characteristics make it well-suited for a conversational application that require low-latency responses from structured tabular data. The system workflow is as follows: when a user poses a query in natural language, the query is used to retrieve the most relevant student record snippets from ChromaDB (using semantic similarity), and those snippets are then provided to the Gemini model. The model, now “augmented” with the retrieved student data, generates a contextual answer – for example, providing the requested student’s information or summary – grounded in the actual records.

To make this solution accessible in practice, we developed a RESTful API using Python Flask that wraps the RAG pipeline. This design allows integration with external platforms or interfaces – for instance, a school’s existing dashboard or a chat frontend – allowing end-users (teachers, administrators, or students) to access the system through existing institutional platforms. The API can handle questions across multiple

categories of data (academic performance, health and medical notes, contact info, demographics, etc.), reflecting the diverse information needs in a school environment.

The research and development of this system followed a Design and Development Research (DDR) Type I model, focusing on the creation and evaluation of a concrete prototype solution. DDR Type I (also known as Type I developmental research) is highly context-specific and centers on designing and assessing a particular educational tool or product [6,7].

In line with this approach, our work iteratively designed the RAG-based interface, then tested it in a controlled scenario to observe its effectiveness and gather feedback. We constructed a mock dataset of 35 student records (with realistic but fictitious data) and crafted 15 representative user queries to evaluate the prototype's performance.

This allowed us to verify that the system could correctly retrieve and present information for a variety of query types (from simple factual questions to more complex or analytical queries) and to ensure that the natural language responses were accurate and contextually grounded in the underlying dataset.

B. Research Objectives

This study aims to design, develop, and evaluate an AI-powered chatbot system that enables natural language access to student data stored in Google Sheets. The specific objectives are as follows:

1) Design Phase:

To investigate user requirements related to student information retrieval from Google Sheets, and to define the functional and technical specifications for a chatbot system that supports natural language queries.

2) Development Phase:

To build a prototype chatbot system that integrates the Retrieval-Augmented Generation (RAG) technique with a RESTful API, enabling semantic search and natural language response generation based on structured student data.

3) Evaluation Phase:

To evaluate the system's performance using computational metrics, specifically through token efficiency analysis and rubric-based relevance scoring, in order to assess its accuracy and response quality when answering natural language queries.

II. METHODOLOGY

The methodology adopted in this study follows the Design and Development Research (DDR) Type I model, as proposed by Richey and Klein [7]. This approach emphasizes the systematic design, development, and evaluation of an educational technology solution. The research process is structured around the three core phases of DDR, as detailed below.

A. Design Phase

The process includes the following steps:

1) User Needs Analysis:

A contextual analysis was conducted to identify challenges faced by non-technical users when retrieving student information. To gather insights, the research team conducted

targeted interviews with a purposive sample comprising five students, five teachers, and five school administrators from the researchers' affiliated institutions. Key issues identified included difficulty accessing specific records, limited spreadsheet literacy, and inefficiencies in locating up-to-date information. These findings were used to inform the system's design requirements.

2) System Architecture Designing

Based on the findings from the user needs analysis, the system was designed using a Retrieval-Augmented Generation (RAG) architecture to support natural language queries over structured student data. The architecture comprises the following core components:

- **Data Embedding:** Each student record from the Google Sheet is first converted into a natural language "chunk" and embedded into vector form using Gemini 1.5 Flash (embedding-001). This enabling semantic similarity retrieval based on conceptual meaning rather than keyword matching.
- **Vector Storage and Retrieval:** The embeddings are stored in a ChromaDB vector database, which enables efficient approximate nearest neighbor search. When a user submits a query, the system retrieves the most relevant student records based on semantic proximity.
- **Language Generation:** Retrieved records are compiled into a prompt, and a response is generated using Gemini 1.5 Flash (chat model). This step ensures that answers are both context-aware and presented in natural, user-friendly language.
- **API Layer:** A RESTful API built with Python Flask enables external systems to interact with the chatbot. Queries are submitted via HTTP POST requests to an /ask endpoint, and responses are returned in structured JSON format, making it easy to integrate the chatbot into existing school information systems.

This architectural design was chosen for its modularity, low latency, and compatibility with educational environments where lightweight, web-based tools are preferable. Its modular structure allows each component to be independently developed, maintained, or upgraded.

B. Development Phase

The system was developed following a structured six-step Retrieval-Augmented Generation (RAG) workflow, enabling natural language access to structured student records stored in Google Sheets. The implementation uses Python 3.10, the Flask web framework for serving the API, Gemini 1.5 Flash for both embedding and generation, and ChromaDB for vector storage and retrieval. Each step is outlined below:

1) Ingest (Data Fetching and Change Detection)

The system retrieves the latest student data from a Google Sheet using a public CSV export URL. A SHA-256 hash is computed from the file content and compared with the previous version stored locally. If no changes are detected, the system

reuses the existing ChromaDB collection; otherwise, it regenerates the vector index. This ensures the database stays up to date while avoiding unnecessary re-computation.

2) *Chunk (Textual Representation of Structured Data)*

Each row in the CSV is converted into a natural language chunk summarizing student details, such as name, ID, class, GPA, parent contact, and medical alerts. This conversion enables the use of language-based embeddings by transforming structured data into meaningful text inputs.

3) *Embed (Semantic Vector Generation)*

Each text chunk is embedded into a high-dimensional vector using the “embedding-001” model from Gemini 1.5 Flash, via the `genai.embed_content()` function. While the embedding model does not natively support token counting, the system uses the `chat_model.count_tokens()` method to track token usage for resource analysis. This dual use of Gemini ensures semantic consistency between embedded records and later response generation.

4) *Store (Vector Database with Metadata)*

The generated embeddings and their associated metadata are stored in a ChromaDB persistent collection. ChromaDB performs approximate nearest neighbor search internally, optimizing for fast semantic similarity retrieval. This allows the system to scale efficiently while supporting low-latency query responses.

5) *Query (Embedding the Question and Retrieving Results)*

When a user submits a question via the /ask RESTful endpoint, the system first embeds the query using the same Gemini model employed during data preparation. This embedded query vector is then passed to ChromaDB, which retrieves the top-K most semantically similar student records based on vector proximity.

The API design follows REST principles, enabling not only manual use (e.g., via curl or Postman) but also seamless integration into external applications, such as learning management systems (LMS), school dashboards, web portals, or messaging platforms. This makes the chatbot adaptable to a wide range of real-world educational contexts.

6) *Response (Prompt Construction and Language Generation)*

The retrieved records are combined into a contextual prompt and sent to the Gemini 1.5 Flash chat model, which generates a fluent and contextually relevant response. The output is returned in JSON format, containing a structured response in a designated answer field.

The system is designed to support both interactive use and backend service use, where other software agents or frontend applications can consume the response programmatically. Robust error handling ensures the API remains stable and informative under edge cases, such as empty results or malformed queries, enhancing its reliability for production-grade integration. This includes comprehensive validation of user input, informative error messages, automatic retries for transient failures, and fallback mechanisms to handle incomplete or ambiguous requests.

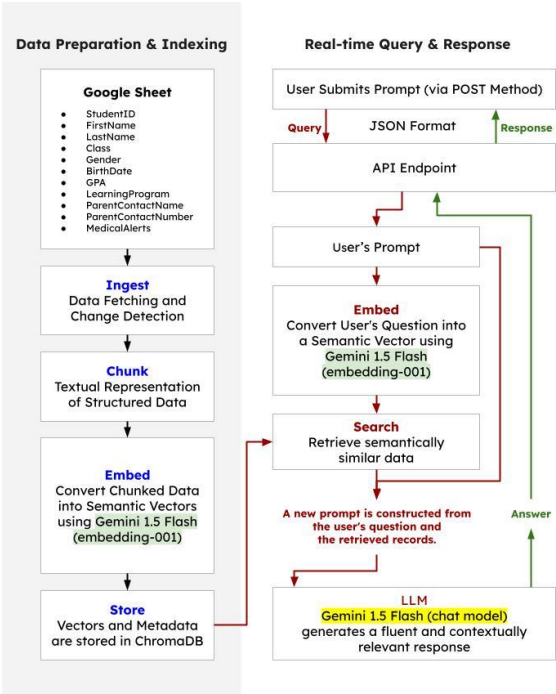


Fig. 1. RAG system workflow.

C. *Evaluation Phase*

This phase focuses on assessing the chatbot’s functionality and effectiveness in responding to typical user queries. The evaluation is divided into two parts as follows:

1) *Test Case:*

To simulate real-world usage, a set of 15 natural language questions was constructed, representing typical queries that teachers, school staff, or administrators might ask when working with student records. These questions were designed to comprehensively evaluate the chatbot’s functionality across five key dimensions, as shown in Table I.

TABLE I. CATEGORIES OF EVALUATION QUESTIONS

Category	Purpose	Questions Items
Academic Records	Test retrieval of structured academic data	1) What is __’s GPA? 2) Which class is __ in? 3) What program is __ enrolled in?
Health Safety	Evaluate response to sensitive info and missing field cases	4) Does __ have any medical alerts? 5) Are there students with allergies? 6) Who has no medical conditions listed?
Parental Contact	Check formatting, number parsing, and name matching	7) Who is the parent of __? 8) What’s __ phone number? 9) Give me the emergency contact for __.
Personal Information	Test parsing of demographic and identity data	10) What is __’s date of birth? 11) What gender is __? 12) When was __ born?
Natural Summarization	Evaluate ability to return full or partial student profiles	13) Tell me everything about __. 14) Show me the details of student __. 15) Summarize __’s student profile.

These questions were selected to cover all critical fields in the dataset – the column structure and example values is shown in Table II. They also represent a mix of direct lookups, semantic retrieval, and freeform summarization tasks.

TABLE II. DATASET ON GOOGLE SHEET

Column Name	Data Type	Example Value
StudentID	Integer	12001
FirstName	String	Chanakan
LastName	String	Sutham
Class	String	M6/1
Gender	String	Female
BirthDate	Date	2007-05-14
GPA	Float	3.85
LearningProgram	String	Science-Math
ParentContactName	String	Somsak Sutham
ParentContactNumber	String	0812345678
MedicalAlerts	String	Peanut Allergy

2) Evaluation Metrics:

Two key evaluation metrics were used to assess system performance: token efficiency and rubric-based relevance scoring.

a) Token Efficiency:

The number of tokens used per interaction—including both the prompt (retrieved content and user query) and the chatbot's response—was measured using the *count_tokens()* method from Gemini 1.5 Flash. This metric reflects the computational cost associated with each query and is important for assessing the system's scalability and cost-effectiveness.

b) Relevance Score:

To evaluate the accuracy and quality of the chatbot's responses, Each answer was independently rated by three evaluators using a standardized 3-point rubric, which assessed how well the response matched the user's question in terms of correctness, completeness, and clarity. The rubric levels were defined as High, Medium, and Low, corresponding to scores of 3, 2, and 1, respectively. The detailed criteria for each score level, along with example scenarios, are presented in Table III.

TABLE III. RELEVANCE SCORE ASSESSMENT

Score	Level	Description	Example Q&A
3	High	The answer is correct, complete, and clearly stated.	Q: What is Thanapol's GPA? A: Thanapol's GPA is 3.75.
2	Medium	The answer is partially correct but lacks key information or clarity.	Q: Show details of Oranong. A: Oranong is in M1/2. (Missing GPA and contact info)
1	Low	The answer is incorrect, irrelevant, or unclear.	Q: Who is Jaruwan's parent? A: Jaruwan was born on June 24. (Not relevant)

The evaluation used a mock dataset of 35 anonymized student records created in Google Sheets. The dataset includes diverse entries across fields such as GPA, gender, learning programs, and medical alerts. Some records were intentionally designed to test edge cases, such as missing fields or similar names, to evaluate the chatbot's robustness under real-world conditions.

III. RESEARCH RESULTS

This section presents the research findings in alignment with the study's objectives. The results focus on system design validation, performance evaluation of the chatbot prototype, and its effectiveness in handling realistic user queries.

A. Fulfillment of System Design Objectives

The prototype chatbot system was successfully developed in alignment with the user requirements identified through contextual interviews with students, teachers, and administrative personnel. The system enables natural language queries related to academic records, health information, parental contacts, and student demographics. It operates through a streamlined Retrieval-Augmented Generation (RAG) pipeline, comprising semantic embedding, ChromaDB-based vector retrieval, and natural language response generation powered with natural language generation powered by the Gemini 1.5 Flash model.

To ensure practical deployment and integration, the chatbot also features a RESTful API built with Flask, enabling seamless integration with external systems such as learning management systems (LMS), school dashboards, and messaging platforms. This API-based architecture enhances the system's adaptability, enabling both interactive and programmatic access to student data across diverse educational contexts.

B. Evaluation of Retrieval-Augmented Chatbot Performance

To evaluate the system's effectiveness, 15 representative natural language queries (Table I) were constructed to reflect authentic information needs commonly encountered in school settings. The evaluation encompassed five key categories: academic records, health and safety, parental contact, personal information, and natural language summarization.

Each query-response pair was assessed using a standardized three-point rubric, focusing on response accuracy, contextual appropriateness, and practical relevance for real-time educational decision-making. From the 15 test cases, 14 responses were received the highest relevance score (3 out of 3), indicating precise alignment with user intent and accurate retrieval of target information. One response received a low score (1), reflecting a partial mismatch or irrelevant output.

C. Response Accuracy and Relevance Scoring

To evaluate computational efficiency and response quality, each of the 15 chatbot interactions was analyzed in terms of token efficiency and response relevance. Token counts were recorded separately for both the input prompt and the generated answer using the *count_tokens()* method of Gemini 1.5 Flash.

On average, the system consumed 316.5 tokens for prompts and 37.7 tokens for answers, yielding a total average of 354.2 tokens per interaction. The prompt lengths exhibited low variance (range: 313–324 tokens), indicating a consistently structured input template. In contrast, answer lengths ranged from 9 to 89 tokens, reflecting the model's ability to scale responses according to query complexity—ranging from factual queries to comprehensive profile summaries.

Regarding response quality, 14 out of 15 responses received the highest relevance score of 3, while one received a low score of 1 due to partial mismatch. The overall average score was 2.93

out of 3, indicating a high degree of alignment between generated responses and user intent.

TABLE IV. TOKEN CONSUMPTION AND RESPONSE QUALITY ACROSS TEST QUERIES

Item (User Questions)	Prompt Tokens	Answer Tokens	Relevance Score
1) What is Siripong's GPA?	317	15	3
2) Which class is Jirapat in?	313	14	3
3) What program is Kawin enrolled in?	324	15	3
4) Does Arthit have any medical alerts?	322	13	3
5) Are there students with allergies?	316	48	3
6) Who has no medical conditions listed?	317	21	1
7) Who is the parent of Boonsri?	318	16	3
8) What's Sutham phone number?	319	40	3
9) Give me the emergency contact for Yodrak.	315	26	3
10) What is Jirapat's date of birth?	320	23	3
11) What gender is Pimchanok?	318	9	3
12) When was Wichai born?	316	20	3
13) Tell me everything about Srisuwan.	313	89	3
14) Show me the details of student 12031.	315	80	3
15) Summarize Apinya's student profile.	315	87	3

IV. DISCUSSION

The results of this study highlight the practicality of a chatbot system built on Retrieval-Augmented Generation (RAG) to support natural language access to student information. The following discussion focuses on key strengths of the system in terms of cost-effectiveness, scalability, and system integration.

A. Performance–Cost Trade-off

A key advantage of the developed system is its low operational cost while maintaining high response accuracy. By utilizing Google's Gemini 1.5 Flash model, the cost is approximately \$0.075 per million input tokens and \$0.30 per million output tokens (for contexts lower than 128k tokens)[8], which is significantly more cost-effective than high-end models such as ChatGPT (GPT-4), Claude, DeepSeek, or Grok [9].

Unlike these premium models, Gemini Flash prioritizes speed and affordability over deep inferential reasoning. Although not optimized for tasks involving complex reasoning, the model performs reliably in structured information retrieval contexts [10]. In this study, the system achieved a high response accuracy of 93% (14 out of 15 responses rated as "High"), confirming that this trade-off is justified for educational use cases involving factual queries and summarization.

This level of cost-efficiency makes AI adoption feasible in schools operating under limited budgets. The system can be deployed across classrooms and administrative units without incurring prohibitive infrastructure costs. Overall, it strikes an effective balance between performance and cost, tailored to the needs of educational institutions that require accurate yet efficient solutions.

The experimental findings in this study are consistent with the findings reported in the systematic literature review by Bazzi and Gaith [11]. Specifically, the high response accuracy and cost-efficiency of the RAG-based system developed here substantiate the review's conclusions regarding RAG's

potential to enhance large language models through external data integration and information retrieval. The reported success in mitigating hallucinations and optimizing performance at reduced cost directly reflects the core themes highlighted in the review, reinforcing the practical value of RAG for real-world educational applications.

B. Scalability and Deployment Flexibility

In addition to cost-efficiency and accuracy, the system is designed for long-term scalability and seamless integration with educational platforms. It adopts a modular, service-oriented architecture using Python Flask as a RESTful API [12], [13], enabling backend services to operate independently from specific interfaces or data sources.

Such a decoupled architecture supports integration with widely adopted educational tools such as Google Sheets, learning management systems (LMSs), dashboards, and messaging applications, making it suitable for varied infrastructure and future customizations. Flask's component-based architecture supports streamlined updates or replacements of individual modules, such as the vector database, embedding model, or API endpoints, without disrupting the overall system.

For data processing, ChromaDB stores and retrieves high-dimensional vector embeddings, supporting real-time semantic search with low latency and high accuracy, even as the dataset grows. Its optimized approximate nearest neighbor search [14] ensures stable performance under high demand, essential in multi-user environments.

The system also leverages Gemini APIs for embedding and response generation, streamlining the AI pipeline and reducing complexity. Its cloud-based, horizontally scalable architecture enables increased load handling through the allocation of additional resources, without requiring modifications to the core code.

This design aligns with findings by Pérez-Jorge et al. [15], whose systematic review on AI-powered APIs in education underscores the need for modular and interoperable systems to enable real-time monitoring, personalization, and automated feedback. The present system meets these demands and helps address current research gaps in the practical deployment of educational APIs.

V. CONCLUSION

This study presents the design, development, and evaluation of an AI-powered chatbot for accessing student records stored in Google Sheets. Using Retrieval-Augmented Generation (RAG) with Gemini 1.5 Flash and ChromaDB, the system enables semantic search and natural language response generation in a computationally efficient, modular, and scalable manner.

The evaluation results demonstrated a response accuracy of 93%, with low average token usage, indicating strong alignment with user queries and high operational efficiency. Its architecture, implemented using a RESTful API and Python Flask, supports integration flexibility and scalability, rendering it suitable for widespread adoption in resource-constrained educational environments.

Future development could explore several directions to further enhance the system's utility. First, extending support for unstructured or semi-structured data (e.g., student essays, attendance notes) could broaden the scope of queries handled. Second, incorporating multilingual support would enable usage in diverse linguistic contexts, especially in non-English-speaking regions. Lastly, integrating user feedback loops and analytics could improve system adaptability over time, allowing personalized refinement based on actual usage patterns in educational settings. These enhancements would further align the system with evolving educational needs and promote sustainable AI integration in schools.

REFERENCES

- [1] E. U. Reshma and P. C. Remya, "A review of different approaches in natural language interfaces to databases," in *Proc. 2017 Int. Conf. Intell. Sustain. Syst. (ICISS)*, Dec. 2017, pp. 801–804, doi: 10.1109/ISSI.2017.8389287.
- [2] T. Brown et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS 2020)*, vol. 33, 2020, pp. 1877–1901. [Online]. Available: <https://arxiv.org/abs/2005.14165>
- [3] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS 2020)*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [4] Y. Huang and J. Huang, "A survey on retrieval-augmented text generation for large language models," *arXiv preprint arXiv:2404.10981*, Apr. 2024. [Online]. Available: <https://arxiv.org/abs/2404.10981>
- [5] Google Cloud, "Gemini 1.5 Flash" *Generative AI Documentation*, Google Cloud, 2025. [Online]. Available: <https://cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/1-5-flash>
- [6] N. A. Hasbullah, N. Fabil, A. M. Yusoff, and A. S. Azmi, "Adapting a design and development research (DDR) approach in designing the mobile application model for tarannum subject," *Adv. Humanities Contemporary Studies*, vol. 3, no. 2, pp. 193–204, 2022.
- [7] R. C. Richey and J. D. Klein, "Developmental research methods: Creating knowledge from instructional design and development practice," *J. Comput. Higher Educ.*, vol. 16, no. 2, pp. 23–38, 2005, doi: 10.1007/BF02961473.
- [8] Google AI for Developers, "Gemini Developer API Pricing," *Gemini API Docs*, Google. [Online]. Available: <https://ai.google.dev/gemini-api/docs/pricing>
- [9] YourGPT.ai, "LLM Cost Calculator," *YourGPT.ai*. [Online]. Available: <https://yourgpt.ai/tools/openai-and-other-llm-api-pricing-calculator>
- [10] Z. Reza, A. Mazur, M. T. Dugdale, and R. Ray-Chaudhuri, "Small Models, Big Support: A Local LLM Framework for Teacher-Centric Content Creation and Assessment using RAG and CAG," *arXiv preprint arXiv:2506.05925*, Jun. 2025. [Online]. Available: <https://arxiv.org/abs/2506.05925>
- [11] W. Bazzi and M. Gaith, *The Wonders of RAG: Streamlining Knowledge with Advanced Techniques*, Systematic Literature Review Report, 2025. [Online]. Available: <https://www.authorea.com/doi/full/10.22541/au.174197235.57492382>
- [12] K. Blinco, T. Grisby, A. Laird, O. O'Neill, V. Srikanth, and C. Smythe, *Adoption of Service Oriented Architecture (SOA) for Enterprise Systems in Education: Recommended Practices*, IMS Global Learning Consortium, 2009. [Online]. Available: https://www.imsglobal.org/soa/soawpvp1p0/imsSOAWhitePaper_v1p0.html
- [13] L. Brooks and E. M. Chen, "Deploying NLP Models as REST APIs with FastAPI and Flask," May 2025. [Online]. Available: https://www.researchgate.net/publication/392158891_Deploying_NLP_Models_as_REST_APIs_with_FastAPI_and_Flask
- [14] J. Knapp and K. Moberg, *From Data to Descriptions: Efficient Data Retrieval in Autonomous Vehicle Development using Generative AI*, Master's thesis, Chalmers Univ. of Technology, Gothenburg, Sweden, 2024. [Online]. Available: <http://hdl.handle.net/20.500.12380/308923>
- [15] D. Pérez-Jorge, M. C. González-Afonso, A. G. Santos-Álvarez, Z. Plasencia-Carballo, and C. D. L. Á. Perdomo-López, "The impact of AI-driven application programming interfaces (APIs) on educational information management," *Information*, vol. 16, no. 7, p. 540, 2025.